МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: „ Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Мета роботи : Ознайомитися з алгоритмом PageRank та його модифікацією під парадигму MapReduce, навчитися використовувати PySpark для реалізації модифікованого алгоритму PageRank та провести експерименти з великими даними.

Завдання

1. Ознайомлення з алгоритмом PageRank та його модифікацією під парадигму MapReduce. Дослідження основних понять та принципів роботи алгоритму PageRank, а також його модифікації під парадигму MapReduce.

2. Розробка модифікованого алгоритму PageRank за допомогою PySpark та алгоритму MapReduce. Реалізація модифікованого алгоритму PageRank за допомогою PySpark та алгоритму MapReduce, використовуючи підходи, що відповідають парадигмі MapReduce.

3. Проведення експериментів з великими даними (на скільки це можливо). Використання розробленого модифікованого алгоритму PageRank для обробки великих даних та проведення експериментів з різними параметрами алгоритму.

4. Аналіз результатів та порівняння зі стандартним алгоритмом PageRank. Аналіз результатів експериментів та порівняння їх з результатами, отриманими за допомогою стандартного алгоритму PageRank

5. Підготувати протокол з результатами виконання роботи, який має містити наступні пункти:

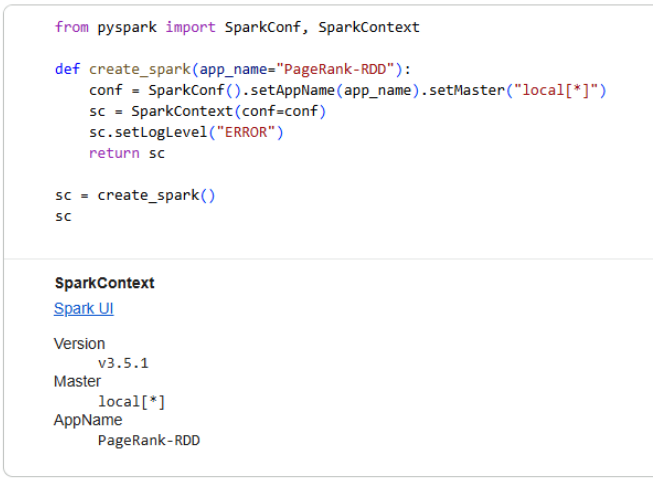
1. Кроки налаштування середовища
2. Запуск та пояснення роботи програми «PageRank» на щонайменше 5-ти різних наборах даних
3. Відповідь на теоретичні запитання

Хід роботи

1. Налаштування серидовища PySpark

Оскільки алгоритм PageRank реалізується у вигляді MapReduce-процесу, то необхідний інструмент, який може працювати з розподіленими даними. Для обробки великих даних (використовуэмо спарк), який дозволяє запускати обчислення у вигляді паралельних задач.

PySpark це Python API до Spark, який дозволяє працювати зі Spark напряму із Python-коду. Також ми створюємо SparkContext це точка входу в Spark-кластер. Саме через SparkContext можна читати файли, створювати розподілені структури даних (RDD) і виконувати MapReduce-алгоритми (SparkContext потрібен для того, щоб реалізований алгоритм PageRank міг працювати в режимі розподілених обчислень)



```
from pyspark import SparkConf, SparkContext

def create_spark(app_name="PageRank-RDD"):
    conf = SparkConf().setAppName(app_name).setMaster("local[*]")
    sc = SparkContext(conf=conf)
    sc.setLogLevel("ERROR")
    return sc

sc = create_spark()
sc
```

SparkContext

Spark UI

Version
v3.5.1
Master
local[*]
AppName
PageRank-RDD

Рис.1 PySpark

2. Реалізація алгоритму PageRank у вигляді MapReduce

Далі я реалізую модифікований алгоритм PageRank, який працює за принципами MapReduce. Алгоритм PageRank імітує поведінку випадкового користувача, який переходить за гіперпосиланнями всередині веб-графа. Ранг кожної сторінки залежить від того, скільки інших сторінок на неї посилається та які ранги мають ці сторінки. Щоб працювати з графом у Spark, нам потрібно виконати такі етапи:

1) load_graph, завантажує текстовий файл та подає його як RDD (це дозволяє розподілено читати дані з файлу)
2) parse_links, перетворює кожний рядок файлу у формат (A, (B, C)), тобто створює структуру вихідних лінків (основа для розрахунку PageRank)
3) initialize_ranks, задає початковий ранг кожній сторінці (у класичному варіанті всі вершини спочатку мають однаковий ранг = 1.0)
4) compute_pagerank, основна частина алгоритму, у якій виконується (Map: сторінка передає свій ранг усім вихідним посиланням, Reduce: ранги, що

надійшли від різних сторінок, сумуються, Damping: моделює ймовірність переходу за випадковим лінком, ітерації повторюються кілька разів для збіжності.

5) run_experiment - пов’язує всі попередні функції, виконує аналіз одного графа та повертає ранги.

Завантаження текстового графа як RDD та парсування рядків файлу у формат (сторінка, список лінків).

```
def load_graph(sc, path):
    return sc.textFile(path)

def parse_links(rdd):
    return (
        rdd.map(lambda line: line.split())
           .map(lambda parts: (parts[0], parts[1:]))
    )
```

Рис.2 Функції завантаження та парсингу

Присвєння кожній сторінці початковий ранг = 1.

```
def initialize_ranks(links):
    return links.mapValues(lambda _: 1.0)
```

Рис.3 Ранг

Основна реалізація алгоритму PageRank у вигляді MapReduce

```
def compute_pagerank(links, ranks, iterations=10, damping=0.85):
    for _ in range(iterations):

        contributions = links.join(ranks).flatMap(
            lambda page_links_rank: [
                (dest, page_links_rank[1][1] / len(page_links_rank[1][0]))
                for dest in page_links_rank[1][0]
            ]
        )

        ranks = (
            contributions.reduceByKey(lambda a, b: a + b)
                         .mapValues(lambda r: (1 - damping) + damping * r)
        )
    return ranks
```

Рис.4 Алгоритм PageRank

Виконання повного запуску алгоритму PageRank для конкретного графа.

```python
def run_experiment(sc, path, iterations=10):
    rdd = load_graph(sc, path)
    links = parse_links(rdd)
    ranks = initialize_ranks(links)

    final_ranks = compute_pagerank(links, ranks, iterations)
    return final_ranks.collect()
```

Рис.5 Функція для запуску алгоритму

3. Експерименти на 5 різних графах

Проведемо аналітику на 5 різних графах.

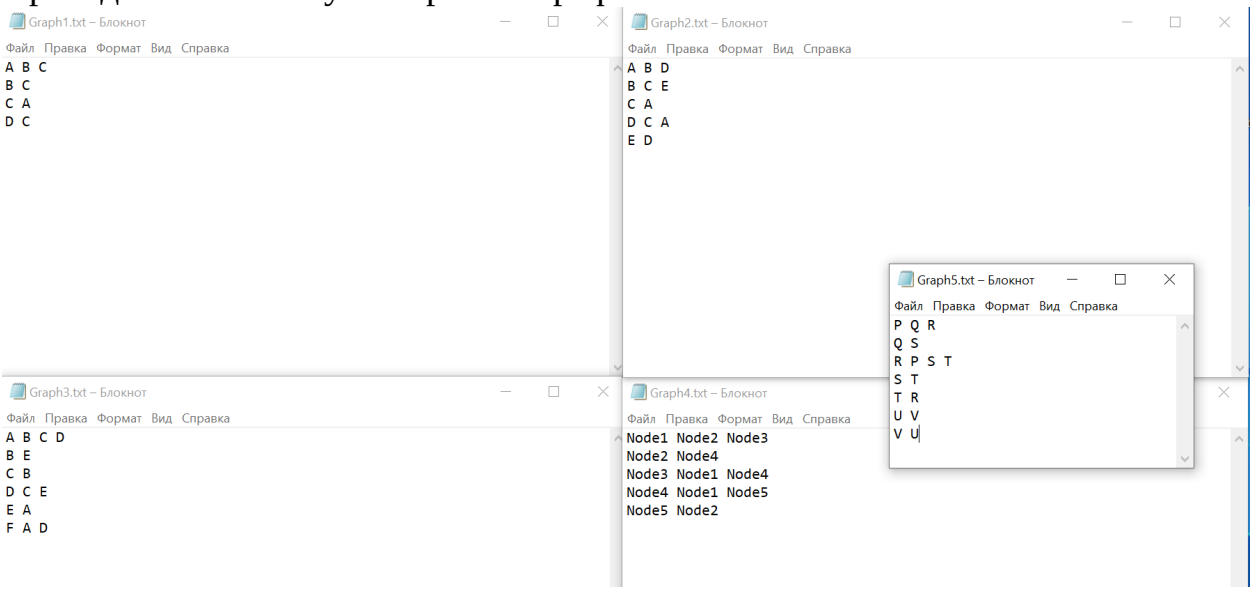


Рис.6 Графи

```
[12]
✓ 4 мин.
```

```python
files = [
    "/content/Graph1.txt",
    "/content/Graph2.txt",
    "/content/Graph3.txt",
    "/content/Graph4.txt",
    "/content/Graph5.txt"
]

for f in files:
    print("\n")
    print("Результати для графа:", f)

    result = run_experiment(sc, f, iterations=10)

    for page, rank in result:
        print(page, round(rank, 4))
```

Рис.7 Виконання експерименту

```
Результати для графа: /content/Graph1.txt
C 1.2638
B 0.6802
A 1.2529


Результати для графа: /content/Graph2.txt
C 1.0
B 0.793
A 1.5131
D 1.207
E 0.4869


Результати для графа: /content/Graph3.txt
C 0.7594
B 1.1816
D 0.5314
A 1.3361
E 1.3884


Результати для графа: /content/Graph4.txt
Node1 0.9899
Node4 1.4397
Node2 1.2442
Node5 0.7434
Node3 0.5827


Результати для графа: /content/Graph5.txt
Q 0.4098
U 1.0
P 0.6058
R 1.6123
S 0.9502
V 1.0
T 1.4219
```

Рис.8 Результат роботи коду

4. Вплив коефіцієнта демпування d

Коефіцієнт демпінгу d є один із ключових параметрів PageRank. Він задає ймовірність того, що користувач продовжить переходити за посиланнями, а не перейде на випадкову сторінку.

1. При d = 0.5 користувач часто «телепортується» ранги вирівнюються.
2. При d = 0.85 отримуємо класичний результат (як у Google).
3. При d = 0.95 ранг найважливіших сторінок росте набагато сильніше.

```
d_values = [0.50, 0.85, 0.95]

graph_path = "/content/Graph1.txt"
for d in d_values:
    print("\n")
    print(f"Результати для d = {d}")

    rdd = load_graph(sc, graph_path)
    links = parse_links(rdd)
    ranks = initialize_ranks(links)

    final_ranks = compute_pagerank(links, ranks, iterations=10, damping=d)

    for page, rank in final_ranks.collect():
        print(page, round(rank, 4))
```

Рис.9 Налаштування коефіцєнтів d

```
Результати для d = 0.5
C 1.1542
B 0.7694
A 1.0774


Результати для d = 0.85
C 1.2638
B 0.6802
A 1.2529


Результати для d = 0.95
C 1.4151
B 0.7228
A 1.4608
```

Рис.10 Результат роботи коду

5. Порівняння MapReduce PageRank та стандартного PageRank

1. Матричний стандартний PageRank (NumPy), це класичний математичний підхід, який працює на малих графах. Результати нормалізуються так, що сума рангів = 1.

[14] ✓ 0 сек.

```python
import numpy as np

def pagerank_standard(graph, d=0.85, iterations=10):
    nodes = list(graph.keys())
    n = len(nodes)

    index = {nodes[i]: i for i in range(n)}

    M = np.zeros((n, n))

    for u in nodes:
        outs = graph[u]
        if len(outs) == 0:
            continue
        for v in outs:
            M[index[v], index[u]] = 1 / len(outs)

    rank = np.ones(n) / n

    for _ in range(iterations):
        rank = (1 - d) / n + d * M @ rank

    return {nodes[i]: rank[i] for i in range(n)}
```

Рис.11 Стандартний PageRank

2. Версія PageRank, реалізована через MapReduce, має іншу нормалізацію рангу (не приводиться до 1), але порядок важливості зберігається. Цей підхід використовується на великих графах, у тому числі в індустрії.

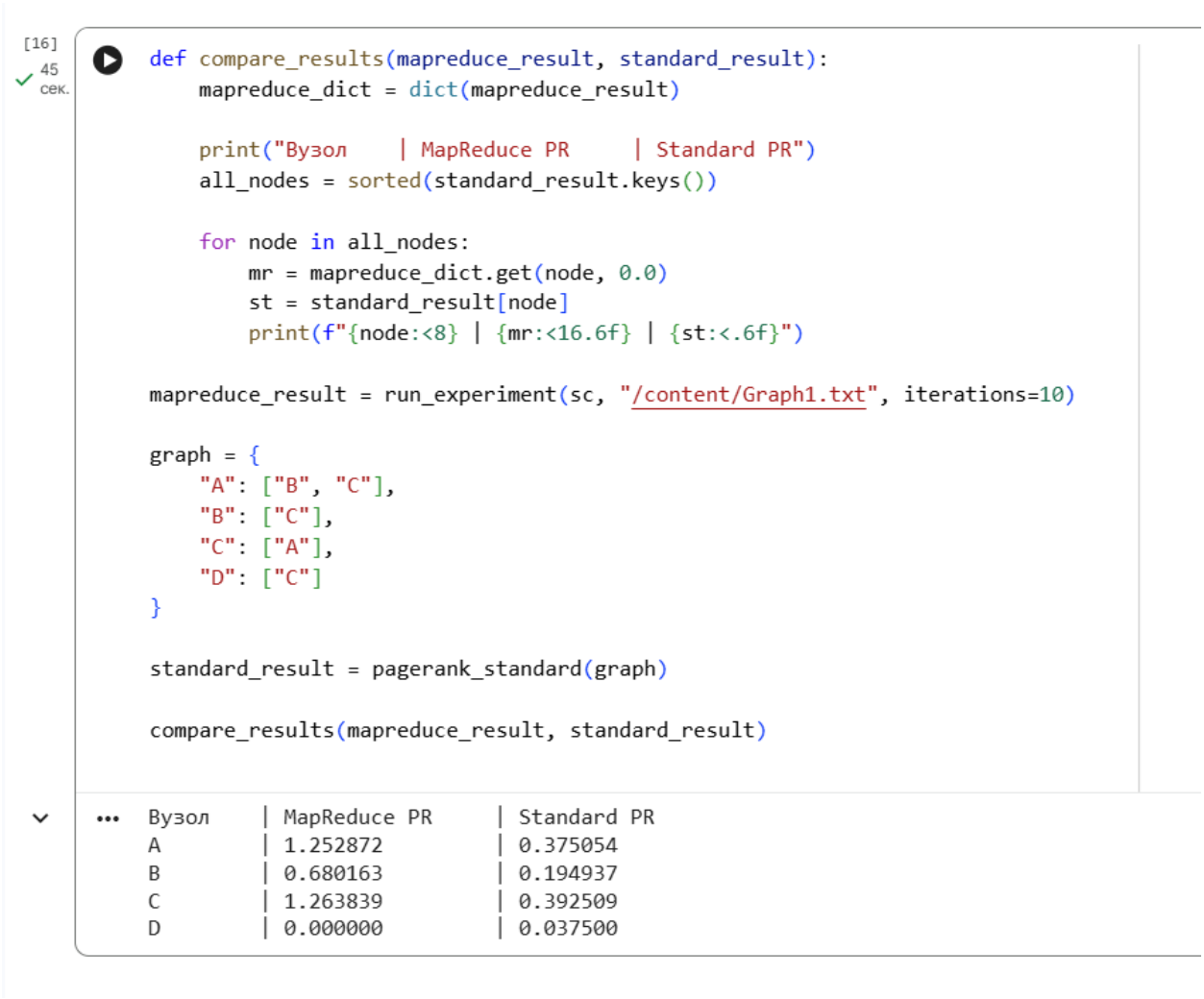


Рис.12 Результат порівняння

Порівняння показує, що обидві версії визначають однаково важливі вузли, але різниця у числових значеннях пов’язана з різною нормалізацією

6. Експерименти з великими даними

Створимо великий синтетичний граф, у якому є: багато сторінок (1000–5000 вузлів), велика кількість посилань, випадкова структура, наближена до реального інтернету

На таких графах стандартний метод (матриця) вже не підходить, оскільки матриця NxN буде занадто великою. MapReduce PageRank, реалізований через PySpark, спеціально створений для таких задач.

```
[17]
✓ 0 сек.
```

```python
import random

def generate_big_graph(path, nodes=5000, edges_per_node=5):
    with open(path, "w") as f:
        for i in range(nodes):
            src = f"N{i}"
            dests = [f"N{random.randint(0, nodes-1)}" for _ in range(edges_per_node)]
            f.write(src + " " + " ".join(dests) + "\n")

generate_big_graph("/content/BigGraph.txt", nodes=5000, edges_per_node=5)
```

Рис.13 Генерація великого графа

Результат роботи коду для великого графа.

```
[18]
✓ 54 сек.
```

```python
big_result = run_experiment(sc, "/content/BigGraph.txt", iterations=10)

# Виведемо перші 20 вузлів з найвищим рангом
sorted_big = sorted(big_result, key=lambda x: x[1], reverse=True)[:20]

print("ТОП-20 найважливіших сторінок великого графа:")
for page, rank in sorted_big:
    print(page, round(rank, 4))
```

```
ТОП-20 найважливіших сторінок великого графа:
N4320 2.9714
N172 2.7464
N2007 2.6736
N3486 2.6085
N500 2.567
N3945 2.5129
N3033 2.4954
N2095 2.465
N2661 2.3991
N3910 2.3826
N2925 2.3798
N1240 2.3408
N3526 2.3315
N2847 2.3259
N3487 2.2777
N598 2.2726
N1382 2.2574
N2852 2.2544
N3456 2.2333
N1756 2.219
```

Рис. 14 Результат роботи коду

Висновок.

У ході виконання лабораторної роботи було реалізовано модифікований алгоритм PageRank у середовищі PySpark, що дало змогу дослідити його роботу в умовах розподілених обчислень та проаналізувати поведінку алгоритму на різних структурах графів. Було проведено експерименти з п’ятьма тестовими графами та одним великим синтетичним графом, що продемонструвало здатність алгоритму коректно визначати найбільш важливі сторінки в мережі та ефективно масштабуватися для обробки значних обсягів даних. Також було порівняно результати MapReduce і класичного матричного PageRank та досліджено вплив коефіцієнта демпінгу, що дозволило сформувати цілісне уявлення про роботу методу та фактори, які визначають розподіл важливості вебсторінок.

Контрольні питання

1. Як визначається важливість веб-сторінок в алгоритмі PageRank?

Важливість веб сторінки в алгоритмі PageRank визначається на основі кількості та якості посилань на неї від інших сторінок. Чим більше сторінок посилається на певну сторінку і чим вищий рейтинг цих сторінок, тим важливішою вона вважається. Алгоритм враховує не тільки сам факт наявності посилань, але й їх вагу, тому сторінка що отримує посилання від впливових вузлів набирає більший рейтинг ніж та, що отримує посилання від малозначущих джерел. Завдяки такому підходу PageRank моделює процес колективної оцінки сторінок усією мережею.

2. Які є основні поняття та принципи роботи алгоритму PageRank?

Основними поняттями алгоритму PageRank є сторінка як вузол графа, гіперпосилання як ребра графа, рейтинг сторінки як числове значення важливості та коефіцієнт демпінгу що моделює поведінку випадкового користувача. Принципи роботи базуються на моделі випадкового переходу за посиланнями де користувач переміщується мережею з урахуванням імовірностей переходів. Алгоритм використовує принцип взаємної залежності коли кожна сторінка отримує свій рейтинг через внесок інших сторінок які на неї посилаються що утворює систему взаємопов'язаних обчислень.

3. Як використовується матриця переходів у алгоритмі PageRank?

Матриця переходів використовується для математичного представлення структури веб графа. У цій матриці кожен стовпець відповідає сторінці з якої здійснюється перехід а кожен рядок представляє сторінку до якої здійснюється перехід. Значення у матриці є ймовірностями переходу які розраховуються як обернені величини кількості вихідних посилань. Множення цієї матриці на вектор рейтингів дозволяє обчислити нові значення важливості сторінок відповідно до поточного розподілу.

4. Як використовується вектор рейтингів у алгоритмі PageRank?

Вектор рейтингів містить поточні значення важливості усіх сторінок у веб графі. На кожній ітерації він оновлюється на основі результатів множення матриці переходів на попередній вектор. Завдяки цьому вектор рейтингів служить основою для наближення до стаціонарного розподілу важливості сторінок. Після завершення ітерацій вектор містить кінцеві ранги які відображають важливість кожної сторінки в мережі.

5. Як використовується ітераційний підхід у алгоритмі PageRank?

Ітераційний підхід дозволяє поступово наблизитися до стабільного розподілу рейтингів. На кожному кроці алгоритм обчислює нові значення рангів на основі попередніх значень. Процес повторюється до тих пір поки зміни у рейтингах не стануть незначними тобто вектор рейтингів не наблизиться до стаціонарного стану. Ітераційність забезпечує збіжність алгоритму навіть при складній структурі веб графа.

6. Як впливає структура веб-графа на реалізацію алгоритму PageRank?

Структура веб графа суттєво впливає на результати PageRank. Якщо сторінка має багато вхідних посилань від важливих сторінок її рейтинг значно зростає. Графи з великою кількістю циклів або зіркоподібними структурами формують групи сторінок що взаємно підсилюють ранги одна одній. Наявність сторінок без вихідних посилань може потребувати додаткової обробки оскільки вони можуть поглинати вагу. У реальних веб графах розподіл посилань нерівномірний і це створює кластеризацію важливих вузлів.

7. Як впливає зміна параметрів алгоритму PageRank на результати його роботи?

Зміна коефіцієнта демпінгу впливає на чутливість алгоритму до структури графа. Низькі значення роблять розподіл рангів більш рівномірним оскільки користувач частіше переходить на випадкову сторінку. Високі значення збільшують різницю між важливими та неважливими сторінками. Також кількість ітерацій впливає на точність результату оскільки недостатня кількість ітерацій може не забезпечити збіжність рейтингу.

8. Які є модифікації алгоритму PageRank та їх вплив на результати роботи алгоритму?

Існує багато модифікацій алгоритму PageRank що адаптуються під різні типи даних і задач. Персоналізований PageRank дозволяє зміщувати вагу у бік певних тематичних сторінок. Topic sensitive PageRank використовує декілька тематичних векторів для визначення значень важливості. Модифікації для соціальних мереж враховують взаємодії між користувачами а не тільки гіперпосилання. Кожна зміна змінює спосіб поширення ваги що може істотно впливати на ранжування.

9. Як використовується алгоритм PageRank в пошукових системах та соціальних мережах?

У пошукових системах PageRank допомагає визначати ступінь авторитетності веб сторінок і впливає на ранжування результатів пошуку. Він дозволяє оцінювати значущість сторінки незалежно від частоти появи ключових слів. У соціальних мережах PageRank використовується для оцінювання впливовості користувачів через аналіз структури їхніх зв’язків. Такі системи можуть визначати важливих лідерів думок або найбільш популярні акаунти.

10. (Додаткове питання) Які є переваги та недоліки алгоритму PageRank порівняно з іншими алгоритмами визначення важливості веб-сторінок?

PageRank може бути застосований у багатьох сферах де об’єкти мають структуру взаємних зв’язків. У біоінформатиці можна аналізувати важливість білків у мережах взаємодій. У фінансових системах PageRank використовується для аналізу ризиковості компаній через мережі транзакцій. У логістичних системах його застосовують для пошуку важливих вузлів транспортних мереж. Алгоритм універсальний тому його можна адаптувати до практично будь якої мережевої структури.

Додаток 1
Посилання на GoogleColab:
https://colab.research.google.com/drive/1xUwLi3Dyg5wx5W3U3RLgPys07nk-L3vY?usp=sharing